

Morphic Studio

Document 28 — Storage & File Management System (Expanded Specification)

Document ID: MS-028

Version: 2.0

Status: Expanded Specification

Priority: Core Platform Infrastructure

Purpose

The Storage & File Management System provides a unified framework for storing, organizing, retrieving, versioning, and protecting every digital asset created or used within Morphe Studio.

It ensures that projects remain organized, reusable, efficient, and resilient throughout their lifecycle.

Rather than treating files as isolated objects, Morphe Studio treats every file as a managed creative asset with metadata, relationships, history, and ownership.

Vision

Creators should never wonder:

- Where is my file?
- Which version is the newest?
- Why did this asset disappear?
- Why was this regenerated?

Everything should be searchable, traceable, reusable, and recoverable.

Core Philosophy

Every creative asset is permanent until intentionally removed.

Files are not disposable outputs—they are reusable building blocks that contribute to long-term creative productivity.

Objectives

The Storage System shall:

- Store all project assets.
 - Organize assets logically.
 - Prevent unnecessary duplication.
 - Support version history.
 - Enable fast retrieval.
 - Maintain asset relationships.
 - Protect data integrity.
 - Scale with future growth.
-

Asset Categories

Supported asset types include:

Project Assets

- Scripts
 - Story Bible files
 - Creative Memory records
 - Episode data
 - Scene definitions
-

Character Assets

- Character sheets
 - Portraits
 - Expressions
 - Outfits
 - Pose references
 - Character metadata
-

Environment Assets

- Locations
- Maps

- Backgrounds
 - Buildings
 - Weather presets
-

Prop Assets

- Weapons
 - Furniture
 - Vehicles
 - Devices
 - Accessories
-

Comic Assets

- Panels
 - Pages
 - Speech bubbles
 - Effects
 - Layout templates
-

Animation Assets

- Timelines
 - Camera paths
 - Motion presets
 - Lip-sync data
 - Animation rigs
 - Scene transitions
-

Audio Assets

- Voice recordings
 - AI-generated voices
 - Sound effects
 - Ambient audio
 - Music
 - Subtitle files
-

Export Assets

- PDFs

- Videos
 - Images
 - Project packages
 - Backup archives
-

Asset Metadata

Every stored asset includes:

- Asset ID
 - Name
 - Type
 - Owner
 - Project
 - Creation Date
 - Modification Date
 - Version
 - File Size
 - Format
 - Status
 - Tags
 - Relationships
 - Source
 - AI Model Used (if applicable)
-

Asset Relationships

Assets may reference:

Characters



Locations



Scenes



Episodes

↓

Projects

↓

Exports

Relationships allow one asset to be reused without duplication.

Folder Organization

Logical organization:

Workspace

↓

Project

↓

Episode

↓

Scene

↓

Asset Category

↓

Individual Asset

The platform manages this automatically while allowing creators to browse intuitively.

Version Control

Every significant modification creates a new version.

Version history records:

- Version Number
- Author
- Timestamp
- Summary of Changes
- Related Workflow
- Approval Status

Users can restore previous versions.

Asset Reuse

Before generating new content, the system checks whether an appropriate asset already exists.

Examples:

- Existing character
- Existing outfit
- Existing location
- Existing voice
- Existing prop

If available, reuse is preferred over regeneration unless the creator requests a variation.

Search & Discovery

Creators can search by:

- Name
- Tag
- Asset Type
- Character
- Project
- Scene
- Date
- Creator
- Status
- AI Model

Future versions may include semantic search using AI.

Storage Optimization

The platform should:

- Detect duplicate assets.
- Compress archives where appropriate.
- Remove temporary files.
- Cache frequently accessed assets.
- Preserve originals.

Optimization should never compromise creator data.

Backup Strategy

Supported backups include:

- Automatic project backups
- Manual backups
- Incremental backups
- Full workspace backups
- Export archives

Backups should be recoverable and verifiable.

User Stories

As a creator, I want every asset I generate to be stored safely.

As a creator, I want to reuse characters and environments without regenerating them.

As a creator, I want to restore earlier versions of my work.

As a creator, I want to quickly find any asset across my projects.

Functional Requirements

The Storage System shall allow users to:

- Upload assets.
- Download assets.

- Organize assets.
 - Search assets.
 - Tag assets.
 - Rename assets.
 - Archive assets.
 - Restore versions.
 - Delete assets (with safeguards).
 - Recover deleted assets where possible.
-

UI Components

The interface includes:

- Asset Browser
 - Project Explorer
 - Search Bar
 - Version History
 - Metadata Panel
 - Storage Dashboard
 - Tag Manager
 - Trash / Recovery Bin
 - Backup Manager
-

Database Entities

Core entities include:

- Asset
 - AssetVersion
 - AssetRelationship
 - AssetTag
 - StorageLocation
 - Backup
 - Archive
 - FileReference
 - MetadataRecord
-

API Considerations

Potential endpoints include:

- Upload Asset
 - Retrieve Asset
 - Search Assets
 - Update Metadata
 - Create Version
 - Restore Version
 - Delete Asset
 - Recover Asset
 - Create Backup
 - Download Package
-

Non-Functional Requirements

The Storage System should:

- Scale efficiently.
 - Preserve data integrity.
 - Minimize duplication.
 - Support large creative libraries.
 - Enable rapid retrieval.
 - Remain resilient against failures.
-

Error Handling

If storage becomes unavailable:

- Queue uploads.
- Preserve temporary work.
- Retry automatically.
- Notify the creator.

If an asset becomes corrupted:

- Detect integrity issues.
 - Restore from the latest valid version or backup.
 - Record the incident.
-

Future Expansion

Future versions may include:

- Distributed object storage.
 - Content delivery network (CDN) integration.
 - AI-powered asset recommendations.
 - Semantic asset search.
 - Automatic archival policies.
 - Cross-project asset libraries.
 - Studio-wide shared repositories.
 - Offline synchronization.
-

Acceptance Criteria

The Storage & File Management System succeeds when creators can trust that every creative asset is organized, discoverable, reusable, versioned, and protected throughout the entire production lifecycle.

Key Principle

Creative assets are long-term investments. Morphic Studio preserves, organizes, and connects every asset so that creators spend more time creating and less time searching, rebuilding, or recovering lost work.

End of Document 28